C++ Standard Library Issues to be moved in Kona

Doc. no. P1457R0

Date: Revised 2019-01-21 at 04:35:38 UTC

Project: Programming Language C++
Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

3012(i). `atomic<T>` is unimplementable for non-`is_trivially_copy_constructible T`

Section: 30.7 [atomics.types.generic] **Status:** Ready **Submitter:** Billy O'Neal III **Opened:** 2017-08-16 **Last modified:** 2018-11-11

Priority: 2

View all other issues in [atomics.types.generic].

View all issues with Ready status.

Discussion:

30.7 [atomics.types.generic] requires that `T` for `std::atomic` is trivially copyable. Unfortunately, that's not sufficient to implement `atomic`. Consider `atomic<T>::load`, which wants to look something like this:

```
template<class T>
struct atomic {
  __compiler_magic_storage_for_t storage;

  T load(memory_order = memory_order_seq_cst) const {
    return __magic_intrinsic(storage);
  }

};
```

Forming this return statement, though, requires that `T` is copy constructible — trivially copyable things aren't necessarily copyable! For example, the following is trivially copyable but breaks libc++, libstdc++, and msvc++:

```
struct NonAssignable {
  int i;
  NonAssignable() = delete;
  NonAssignable(int) : i(0) {}
  NonAssignable(const NonAssignable&) = delete;
  NonAssignable(NonAssignable&&) = default;
  NonAssignable& operator=(const NonAssignable&) = delete;
  NonAssignable& operator=(NonAssignable&&) = delete;
  ~NonAssignable() = default;
};
```

All three standard libraries are happy as long as `T` is trivially copy constructible, assignability is not required. Casey Carter says that we might want to still require trivially copy assignable though, since what happens when you do an `atomic<T>::store` is morally an "assignment" even if it doesn't use the user's assignment operator.

[2017-11 Albuquerque Wednesday issue processing]

Status to Open; Casey and STL to work with Billy for better wording.

Should this include trivially copyable as well as trivially copy assignable?

2017-11-09, Billy O'Neal provides updated wording.

Previous resolution [SUPERSEDED]:

> This resolution is relative to N4687.
>
> 1. Edit 30.7 [atomics.types.generic] as indicated:
>
> > -1- If `is_trivially_copy_constructible_v<T>` is `false`, the program is ill-formed~~The template argument for `T` shall be trivially copyable (6.7 [basic.types])~~. [*Note:* Type arguments that are not also statically initializable may be difficult to use. — *end note*]

Previous resolution [SUPERSEDED]:

> This resolution is relative to N4687.
>
> 1. Edit 30.7 [atomics.types.generic] as indicated:
>
> > -1- If `is_copy_constructible_v<T>` is `false` or if `is_trivially_copyable_v<T>` is `false`, the program is ill-formed~~The template argument for `T` shall be trivially copyable (6.7 [basic.types])~~. [*Note:* Type arguments that are not also statically initializable may be difficult to use. — *end note*]

[2017-11-12, Tomasz comments and suggests alternative wording]

According to my understanding during Albuquerque Saturday issue processing we agreed that we want the type used with the atomics to have non-deleted and trivial copy/move construction and assignment.

Wording note: `CopyConstructible` and `CopyAssignable` include semantic requirements that are not checkable at compile time, so these are requirements imposed on the user and cannot be validated by an implementation without heroic efforts.

[2018-11 San Diego Thursday night issue processing]

Status to Ready.

Proposed resolution:

This resolution is relative to N4700.

1. Edit 30.7 [atomics.types.generic] as indicated:

> -1- The template argument for `T` shall meet the `CopyConstructible` and `CopyAssignable` requirements. If `is_trivially_copyable_v<T> && is_copy_constructible_v<T> && is_move_constructible_v<T> && is_copy_assignable_v<T> && is_move_assignable_v<T>` is `false`, the program is ill-formed~~be trivially copyable (6.7 [basic.types])~~. [*Note:* Type arguments that are not also statically initializable may be difficult to use. — *end note*]

3040(i). `basic_string_view::starts_with` *Effects* are incorrect

Section: 20.4.2.6 [string.view.ops] **Status:** Tentatively Ready **Submitter:** Marshall Clow **Opened:** 2017-11-29 **Last modified:** 2018-11-11

Priority: 0

View all other issues in [string.view.ops].

View all issues with Tentatively Ready status.

Discussion:

The effects of `starts_with` are described as equivalent to `return compare(0, npos, x) == 0`.

This is incorrect, because it returns `false` when you check to see if any sequence begins with the empty sequence. (There are other failure cases, but that one's easy)

As a drive-by fix, we can make the *Effects:* for `starts_with` and `ends_with` clearer.

Those are the second and proposed third changes, and they are not required.

[2017-12-13 Moved to Tentatively Ready after 8 positive votes for P0 on c++std-lib.]

Previous resolution: [SUPERSEDED]

This wording is relative to N4713.

1. Change 20.4.2.6 [string.view.ops] p20 as indicated:

```
constexpr bool starts_with(basic_string_view x) const noexcept;
```

-20- *Effects:* Equivalent to: `return size() >= x.size() && compare(0, ~~npos~~x.size(), x) == 0;`

2. Change 20.4.2.6 [string.view.ops] p21 as indicated:

```
constexpr bool starts_with(charT x) const noexcept;
```

-21- *Effects:* Equivalent to: `return !empty() && traits::eq(front(), x)~~starts_with(basic_string_view(&x, 1))~~;`

3. Change 20.4.2.6 [string.view.ops] p24 as indicated:

```
constexpr bool ends_with(charT x) const noexcept;
```

-24- *Effects:* Equivalent to: `return !empty() && traits::eq(back(), x)~~ends_with(basic_string_view(&x, 1))~~;`

[2018-01-23, Reopening due to a comment of Billy Robert O'Neal III requesting a change of the proposed wording]

The currently suggested wording has:

> *Effects:* Equivalent to: `return size() >= x.size() && compare(0, x.size(), x) == 0;`

but `compare()` already does the `size() >= x.size()` check.

It seems like it should say:

> *Effects:* Equivalent to: `return substr(0, x.size()) == x;`

[2018-10-29 Moved to Tentatively Ready after 5 positive votes for P0 on c++std-lib.]

Proposed resolution:

This wording is relative to N4713.

1. Change 20.4.2.6 [string.view.ops] p20 as indicated:

```
constexpr bool starts_with(basic_string_view x) const noexcept;
```

-20- *Effects:* Equivalent to: `return substr(0, x.size()) == x~~compare(0, npos, x) == 0~~;`

2. Change 20.4.2.6 [string.view.ops] p21 as indicated:

```
constexpr bool starts_with(charT x) const noexcept;
```

-21- *Effects:* Equivalent to: `return !empty() && traits::eq(front(), x)~~starts_with(basic_string_view(&x, 1))~~;`

3. Change 20.4.2.6 [string.view.ops] p24 as indicated:

```
constexpr bool ends_with(charT x) const noexcept;
```

-24- *Effects:* Equivalent to: `return !empty() && traits::eq(back(), x)~~ends_with(basic_string_view(&x, 1))~~;`

3077(i). `(push|emplace)_back` should invalidate the `end` iterator

Section: 21.3.11.5 [vector.modifiers] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2018-03-10 **Last modified:** 2018-12-03

Priority: 3

View all other issues in [vector.modifiers].

View all issues with Tentatively Ready status.

Discussion:

21.3.11.5 [vector.modifiers] paragraph 1 specifies that `emplace_back` and `push_back` do not invalidate iterators before the insertion point when reallocation is unnecessary:

> *Remarks:* Causes reallocation if the new size is greater than the old capacity. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence. If no reallocation happens, all the iterators and references before the insertion point remain valid. […]

This statement is redundant, given the blanket wording in 21.2.1 [container.requirements.general] paragraph 12:

> Unless otherwise specified (either explicitly or by defining a function in terms of other functions), invoking a container member function or passing a container as an argument to a library function shall not invalidate iterators to, or change the values of, objects within that container.

It seems that this second sentence (1) should be a note that reminds us that the blanket wording applies here when no reallocation occurs, and/or (2) actually intends to specify that iterators at and after the insertion point are invalidated.

Also, it seems intended that reallocation should invalidate the `end` iterator as well.

[2018-06-18 after reflector discussion]

Priority set to 3

Previous resolution [SUPERSEDED]:

> 1. Edit 21.3.11.5 [vector.modifiers] as indicated:
>
> > -1- *Remarks:* Invalidates the past-the-end iterator. Causes reallocation if the new size is greater than the old capacity. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence. [*Note:* If no reallocation happens, all the iterators and references before the insertion point remain valid.*—end note*] If an exception is thrown […]

[2018-11-28 Casey provides an updated P/R]

Per discussion in the prioritization thread on the reflector.

[2018-12-01 Status to Tentatively Ready after seven positive votes on the reflector.]

Proposed resolution:

This wording is relative to the post-San Diego working draft.

1. Change 20.3.2.4 [string.capacity] as indicated:

```
void shrink_to_fit();
```

-11- *Effects:* `shrink_to_fit` is a non-binding request to reduce `capacity()` to `size()`. [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*] It does not increase `capacity()`, but may reduce `capacity()` by causing reallocation.

-12- *Complexity:* If the size is not equal to the old capacity, linear in the size of the sequence; otherwise constant.

-13- *Remarks:* Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. [*Note:* If no reallocation happens, they remain valid. *— end note*]

2. Change 21.3.8.3 [deque.capacity] as indicated:

```
void shrink_to_fit();
```

-5- *Requires:* `T` shall be *`Cpp17MoveInsertable`* into `*this`.

-6- *Effects:* `shrink_to_fit` is a non-binding request to reduce memory use but does not change the size of the sequence. [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations. —*end note*] If the size is equal to the old capacity, or if an exception is thrown other than by the move constructor of a non-*`Cpp17CopyInsertable`* `T`, then there are no effects.

-7- *Complexity:* If the size is not equal to the old capacity, linear in the size of the sequence; otherwise constant.

-8- *Remarks:* ~~`shrink_to_fit`~~ If the size is not equal to the old capacity, then invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator.

3. Change 21.3.11.3 [vector.capacity] as indicated:

```
void reserve(size_type n);
```

[…]

-7- *Remarks:* Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. [*Note:* If no reallocation happens, they remain valid. — *end note*] No reallocation shall take place during insertions that happen after a call to `reserve()` until ~~the time when~~ an insertion would make the size of the vector greater than the value of `capacity()`.

```
void shrink_to_fit();
```

[…]

-10- *Complexity:* If reallocation happens, linear in the size of the sequence.

-11- *Remarks:* Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. [*Note:* If no reallocation happens, they remain valid. ~~—~~ *end note*]

4. Change 21.3.11.5 [vector.modifiers] as indicated:

-1- *Remarks:* Causes reallocation if the new size is greater than the old capacity. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator. If no reallocation happens, ~~all the iterators and references~~ then references, pointers, and iterators before the insertion point remain valid but those at or after the insertion point, including the past-the-end iterator, are invalidated. If an exception is thrown […]

-2- *Complexity:* ~~The complexity is~~ If reallocation happens, linear in the number of elements of the resulting vector; otherwise linear in the number of elements inserted plus the distance to the end of the vector.

3087(i). One final `&x` in §[list.ops]

Section: 21.3.10.5 [list.ops] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2018-03-19 **Last modified:** 2018-11-11

Priority: 3

View other active issues in [list.ops].

View all other issues in [list.ops].

View all issues with Tentatively Ready status.

Discussion:

LWG 3017 missed an instance of `&x` in 21.3.10.5 [list.ops] p14.

[2018-06-18 after reflector discussion]

Priority set to 3

[2018-10-15 Status to Tentatively Ready after seven positive votes on the reflector.]

Proposed resolution:

This wording is relative to N4727.

1. Edit 21.3.10.5 [list.ops] as indicated:

```
void splice(const_iterator position, list& x, const_iterator first,
            const_iterator last);
void splice(const_iterator position, list&& x, const_iterator first,
            const_iterator last);
```

-11- *Requires:* […]

-12- *Effects:* […]

-13- *Throws:* Nothing.

-14- *Complexity:* Constant time if `&addressof(x) == this`; otherwise, linear time.

3101(i). `span`'s `Container` constructors need another constraint

Section: 21.7.3.2 [span.cons] **Status:** Tentatively Ready **Submitter:** Stephan T. Lavavej **Opened:** 2018-04-12 **Last modified:** 2018-11-11

Priority: 1

View all other issues in [span.cons].

View all issues with Tentatively Ready status.

Discussion:

When I overhauled `span`'s constructor constraints, I was careful about the built-in array, `std::array`, and converting `span` constructors. These types contain bounds information, so we can achieve safety at compile-time by permitting implicit conversions if and only if the destination extent is dynamic (this accepts anything by recording the size at runtime) or the source and destination extents are identical. However, I missed the fact that the `Container` constructors are the opposite case. A `Container` (e.g. a `vector`) has a size that's known only at runtime. It's safe to convert this to a `span` with `dynamic_extent`, but for consistency and safety, this shouldn't implicitly convert to a `span` with fixed extent. (The more verbose `(ptr, count)` and `(first, last)` constructors are available to construct fixed extent spans from runtime-length ranges. Note that debug precondition checks are equally possible with the `Container` and `(ptr, count)`/`(first, last)` constructors. The issue is that implicit conversions are notoriously problematic, so they should be permitted only when they are absolutely known to be safe.)

[2018-04-24 Priority set to 1 after discussion on the reflector.]

[2018-06 Rapperswil Thursday issues processing]

Status to LEWG. Should this be ill-formed, or fail at runtime if the container is too small? Discussion on the reflector here.

[2018-11 San Diego Saturday]

LEWG said that they're fine with the proposed resolution. Status to Tentatively Ready.

Proposed resolution:

This wording is relative to N4741.

1. Edit 21.7.3.2 [span.cons] as indicated:

```
template<class Container> constexpr span(Container& cont);
template<class Container> constexpr span(const Container& cont);
```

-14- *Requires:* `[data(cont), data(cont) + size(cont))` shall be a valid range. ~~If `extent` is not equal to `dynamic_extent`, then `size(cont)` shall be equal to `extent`.~~

-15- *Effects:* Constructs a `span` that is a view over the range `[data(cont), data(cont) + size(cont))`.

-16- *Postconditions:* `size() == size(cont) && data() == data(cont)`.

-17- *Throws:* What and when `data(cont)` and `size(cont)` throw.

-18- *Remarks:* These constructors shall not participate in overload resolution unless:

(18.?) — `extent == dynamic_extent`,

(18.1) — `Container` is not a specialization of `span`,

(18.2) — `Container` is not a specialization of `array`,

[…]

3112(i). `system_error` and `filesystem_error` constructors taking a `string` may not be able to meet their postconditions

Section: 18.5.7.2 [syserr.syserr.members], 28.11.8.1 [fs.filesystem_error.members] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2018-05-10 **Last modified:** 2019-01-20

Priority: 0

View other active issues in [syserr.syserr.members].

View all other issues in [syserr.syserr.members].

View all issues with Tentatively Ready status.

Discussion:

The constructors of `system_error` and `filesystem_error` taking a `std::string what_arg` are specified to have a postcondition of `string(what()).find(what_arg) != string::npos` (or the equivalent with `string_view`). This is not possible if `what_arg` contains an embedded null character.

[2019-01-20 Reflector prioritization]

Set Priority to 0 and status to Tentatively Ready

Proposed resolution:

This wording is relative to N4727.

> *Drafting note:* This contains a drive-by editorial change to use `string_view` for these postconditions rather than `string`.

1. Edit 18.5.7.2 [syserr.syserr.members] p1-4 as indicated:

```
system_error(error_code ec, const string& what_arg);
```

-1- *Effects:* Constructs an object of class `system_error`.

-2- *Postconditions:* `code()` == `ec` and `string_view(what()).find(what_arg.c_str()) != string_view::npos`.

```
system_error(error_code ec, const char* what_arg);
```

-3- *Effects:* Constructs an object of class `system_error`.

-4- *Postconditions:* `code() == ec` and `string_view(what()).find(what_arg) != string_view::npos`.

2. Edit 18.5.7.2 [syserr.syserr.members] p7-10 as indicated:

```
system_error(int ev, const error_category& ecat, const std::string& what_arg);
```

-7- *Effects:* Constructs an object of class `system_error`.

-8- *Postconditions:* `code()` == `error_code(ev, ecat)` and `string_view(what()).find(what_arg.c_str()) != string_view::npos`.

```
system_error(int ev, const error_category& ecat, const char* what_arg);
```

-9- *Effects:* Constructs an object of class `system_error`.

-10- *Postconditions:* `code()` == `error_code(ev, ecat)` and `string_view(what()).find(what_arg) != string_view::npos`.

3. Edit 28.11.8.1 [fs.filesystem_error.members] p2-4 as indicated:

```
filesystem_error(const string& what_arg, error_code ec);
```

-2- *Postconditions:*

- `code() == ec`,
- `path1().empty() == true`,
- `path2().empty() == true`, and
- `string_view(what()).find(what_arg.c_str()) != string_view::npos`.

```
filesystem_error(const string& what_arg, const path& p1, error_code ec);
```

-3- *Postconditions:*

- `code() == ec`,
- `path1()` returns a reference to the stored copy of `p1`,
- `path2().empty() == true`, and
- `string_view(what()).find(what_arg.c_str()) != string_view::npos`.

```
filesystem_error(const string& what_arg, const path& p1, const path& p2, error_code ec);
```

-4- *Postconditions:*

- `code() == ec`,
- `path1()` returns a reference to the stored copy of `p1`,
- `path2()` returns a reference to the stored copy of `p2`,
- `string_view(what()).find(what_arg.c_str()) != string_view::npos`.

3119(i). Program-definedness of closure types

Section: 99 [defns.program.defined.spec] **Status:** Ready **Submitter:** Hubert Tong **Opened:** 2018-06-09 **Last modified:** 2018-11-11

Priority: 2

View all issues with Ready status.

Discussion:

The description of closure types in 7.5.5.1 [expr.prim.lambda.closure] says:

> An implementation may define the closure type differently […]

The proposed resolution to LWG 2139 defines a "program-defined type" to be a

> class type or enumeration type that is not part of the C++ standard library and not defined by the implementation, or an instantiation of a program-defined specialization

I am not sure that the intent of whether closure types are or are not program-defined types is clearly conveyed by the wording.

[2018-06-23 after reflector discussion]

Priority set to 2

[2018-08-14 Casey provides additional discussion and a Proposed Resolution]

We use the term "program-defined" in the library specification to ensure that two users cannot create conflicts in a component in namespace `std` by specifying different behaviors for the same type. For example, we allow users to specialize `common_type` when at least one of the parameters is a program-defined type. Since two users cannot define the same program-defined type, this rule prevents two users (or libraries) defining the same specialization of `std::common_type`.

Since it's guaranteed that even distinct utterances of identical lambda expressions produce closures with distinct types (7.5.5.1 [expr.prim.lambda.closure]), adding closure types to our term "program-defined type" is consistent with the intended use despite that such types are technically defined by the implementation.

Previous resolution [SUPERSEDED]:

This wording is relative to N4762.

- Modify 15.3.20 [defns.prog.def.type] as follows:

> **program-defined type**
> class type or enumeration type that is not part of the C++ standard library and not defined by the implementation (except for closure types (7.5.5.1 [expr.prim.lambda.closure]) for program-defined lambda expressions), or an instantiation of a program-defined specialization

[2018-08-23 Batavia Issues processing]

Updated wording

[2018-11 San Diego Thursday night issue processing]

Status to Ready.

Proposed resolution:

This wording is relative to N4762.

- Modify 15.3.20 [defns.prog.def.type] as follows:

> **program-defined type**
> non-closure class type or enumeration type that is not part of the C++ standard library and not defined by the implementation, or a closure type of a non-implementation-provided lambda expression, or an instantiation of a program-defined specialization

---

3133(i). Modernizing numeric type requirements

Section: 25.2 [numeric.requirements] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2018-07-05 **Last modified:** 2019-01-20

Priority: 0

View all other issues in [numeric.requirements].

View all issues with Tentatively Ready status.

Discussion:

25.2 [numeric.requirements] contains some very old wording that hasn't been changed since C++98 except for issue 2699. As a result, it is at once over- and under-restrictive. For example:

- It requires a class to have various member functions, but not that said member functions be actually callable (non-ambiguous, non-deleted) or called by the applicable operations;
- It doesn't disallow function or array types;
- It disallows *ref-qualifier*s on the assignment operator by restricting the signature;
- It bans unary `operator&` for class types, but not enumeration types; and
- It tries to impose semantic equivalence requirements on `T`'s special member functions, but doesn't require that copying produces an equivalent value to the original.

We can significantly clean up this wording by using the existing named requirements. For ease of review, the following table provides a side-by-side comparison of the current and proposed wording.

Before	After
A C++ program shall instantiate these components only with a type `T` that satisfies the following requirements: [*Footnote* …]	A C++ program shall instantiate these components only with a cv-unqualified object type `T` that satisfies the *Cpp17DefaultConstructible*, *Cpp17CopyConstructible*, *Cpp17CopyAssignable*, and *Cpp17Destructible*~~following~~ requirements (15.5.3.1 [utility.arg.requirements]).~~:~~
(1.1) — `T` is not an abstract class (it has no pure virtual member functions);	*Cpp17DefaultConstructible*
(1.2) — `T` is not a reference type; (1.3) — `T` is not cv-qualified;	Implied by "cv-unqualified object type"
(1.4) — If `T` is a class, it has a public default constructor;	*Cpp17DefaultConstructible*
(1.5) — If `T` is a class, it has a public copy constructor with the signature `T::T(const T&)`;	*Cpp17CopyConstructible*

(1.6) — If `T` is a class, it has a public destructor;	*Cpp17Destructible*
(1.7) — If `T` is a class, it has a public copy assignment operator whose signature is either `T& T::operator=(const T&)` or `T& T::operator=(T);`	*Cpp17CopyAssignable*
(1.8) — If `T` is a class, its assignment operator, copy and default constructors, and destructor shall correspond to each other in the following sense: (1.8.1) — Initialization of raw storage using the copy constructor on the value of `T()`, however obtained, is semantically equivalent to value-initialization of the same raw storage. (1.8.2) — Initialization of raw storage using the default constructor, followed by assignment, is semantically equivalent to initialization of raw storage using the copy constructor. (1.8.3) — Destruction of an object, followed by initialization of its raw storage using the copy constructor, is semantically equivalent to assignment to the original object. [*Note:* […] — *end note*]	These requirements are implied by *Cpp17CopyConstructible* and *Cpp17CopyAssignable*'s requirement that the value of the copy is equivalent to the source.
(1.9) — If `T` is a class, it does not overload unary `operator&`.	*omitted now that we have `std::addressof`*

[2019-01-20 Reflector prioritization]

Set Priority to 0 and status to Tentatively Ready

Proposed resolution:

This wording is relative to the post-Rapperswil 2018 working draft.

1. Edit 25.2 [numeric.requirements] p1 as indicated, striking the entire bulleted list:

> -1- The `complex` and `valarray` components are parameterized by the type of information they contain and manipulate. A C++ program shall instantiate these components only with a cv-unqualified object type `T` that satisfies the *Cpp17DefaultConstructible*, *Cpp17CopyConstructible*, *Cpp17CopyAssignable*, and *Cpp17Destructible*~~following~~ requirements (15.5.3.1 [utility.arg.requirements]). ~~:~~[*Footnote:* …]
>
> ~~(1.1) — `T` is not an abstract class (it has no pure virtual member functions);~~
>
> ~~[…]~~
>
> ~~(1.9) — If `T` is a class, it does not overload unary `operator&`.~~

2. Edit 25.7.2.4 [valarray.access] p3-4 as indicated:

```
const T&  operator[](size_t n) const;
T& operator[](size_t n);
```

> -1- *Requires:* `n < size()`.
>
> -2- *Returns:* […]

-3- *Remarks*: The expression `&addressof(a[i+j]) == &addressof(a[i]) + j` evaluates to `true` for all `size_t i` and `size_t j` such that `i+j < a.size()`.

-4- The expression `&addressof(a[i]) != &addressof(b[j])` evaluates to `true` for any two arrays `a` and `b` and for any `size_t i` and `size_t j` such that `i < a.size()` and `j < b.size()`. [*Note:* […] — *end note*]

3144(i). `span` does not have a `const_pointer` typedef

Section: 21.7.3.1 [span.overview] **Status:** Tentatively Ready **Submitter:** Louis Dionne **Opened:** 2018-07-23 **Last modified:** 2019-01-20

Priority: 0

View all other issues in [span.overview].

View all issues with Tentatively Ready status.

Discussion:

`std::span` does not have a typedef for `const_pointer` and `const_reference`. According to Marshall Clow, this is merely an oversight.

[2019-01-20 Reflector prioritization]

Set Priority to 0 and status to Tentatively Ready

Proposed resolution:

This wording is relative to N4750.

1. Change 21.7.3.1 [span.overview], class template `span` synopsis, as indicated:

```
namespace std {
  template<class ElementType, ptrdiff_t Extent = dynamic_extent>
  class span {
  public:
    // constants and types
    using element_type = ElementType;
    using value_type = remove_cv_t<ElementType>;
    using index_type = ptrdiff_t;
    using difference_type = ptrdiff_t;
    using pointer = element_type*;
    using const_pointer = const element_type*;
    using reference = element_type&;
    using const_reference = const element_type&;
    using iterator = implementation-defined;
    using const_iterator = implementation-defined;
    using reverse_iterator = reverse_iterator<iterator>;
    using const_reverse_iterator = reverse_iterator<const_iterator>;
    static constexpr index_type extent = Extent;

    […]
  };
  […]
}
```

3173(i). Enable CTAD for *`ref-view`*

Section: 23.8.3.1 [range.view.ref] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2018-12-09 **Last modified:** 2019-01-20

Priority: 0

View all issues with Tentatively Ready status.

Discussion:

In the specification of `view::all` in 23.8.3 [range.all], paragraph 2.2 states that `view::all(E)` is sometimes expression-equivalent to "*`ref-view`*`{E}` if that expression is well-formed". Unfortunately, the expression *`ref-view`*`{E}` is **never** well-formed: *`ref-view`*'s only non-default constructor is a perfect-forwarding-ish constructor template that accepts only arguments that convert to lvalues of the *`ref-view`*'s template argument type, and either do not convert to rvalues or have a better lvalue conversion (similar to the `reference_wrapper` converting constructor (19.14.5.1 [refwrap.const]) after issue 2993).

Presumably this breakage was not intentional, and we should add a deduction guide to enable class template argument deduction to function as intended by paragraph 2.2.

[2018-12-16 Status to Tentatively Ready after six positive votes on the reflector.]

Proposed resolution:

This wording is relative to N4791.

1. Modify the *`ref-view`* class synopsis in [ranges.view.ref] as follows:

```
namespace std::ranges {
  template<Range R>
    requires is_object_v<R>
  class ref-view : public view_interface<ref-view<R>> {
    […]
  };

  template<class R>
    ref_view(R&) -> ref_view<R>;
}
```

3179(i). `subrange` should always model `Range`

Section: 23.6.3.1 [range.subrange.ctor] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2018-12-21 **Last modified:** 2019-01-20

Priority: 0

View all issues with Tentatively Ready status.

Discussion:

The constructors of `subrange` place no requirements on the iterator and sentinel values from which a `subrange` is constructed. They allow constructions like `subrange{myvec.end(), myvec.begin()}` in which the resulting `subrange` isn't in the domain of the `Range` concept which requires that "`[ranges::begin(r), ranges::end(r))` denotes a range" ([range.range]/3.1). We should forbid the construction of abominable values like this to enforce the useful semantic that values of `subrange` are always in the domain of `Range`.

(From ericniebler/stl2#597.)

[2019-01-20 Reflector prioritization]

Set Priority to 0 and status to Tentatively Ready

Proposed resolution:

This wording is relative to N4791.

1. Change 23.6.3.1 [range.subrange.ctor] as indicated:

```
constexpr subrange(I i, S s) requires (!StoreSize);
```

-?- *Expects:* `[i, s)` is a valid range.

-1- *Effects:* Initializes `begin_` with `i` and `end_` with `s`.

```
constexpr subrange(I i, S s, iter_difference_t<I> n)
  requires (K == subrange_kind::sized);
```

-2- *Expects:* `[i, s)` is a valid range, and `n == ranges::distance(i, s)`.

3180(i). Inconsistently named return type for `ranges::minmax_element`

Section: 24.4 [algorithm.syn] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2018-12-21 **Last modified:** 2019-01-20

Priority: 0

View all issues with Tentatively Ready status.

Discussion:

The overloads of `std::ranges::minmax_element` are specified to return `std::ranges::minmax_result`, which is inconsistent with the intended design. When an algorithm `foo` returns an aggregate of multiple results, the return type should be named `foo_result`. The spec should introduce an alias `minmax_element_result` for `minmax_result` and use that alias as the return type of the `std::ranges::minmax_element` overloads.

[2019-01-11 Status to Tentatively Ready after five positive votes on the reflector.]

During that reflector discussion several contributers questioned the choice of alias templates to denote algorithm result types or particular apsects of it. Since this approach had been approved by LEWG before, it was suggested to those opponents to instead write a paper, because changing this as part of this issue would be a design change that would require a more global fixing approach.

Proposed resolution:

This wording is relative to N4791.

1. Change 24.4 [algorithm.syn] as indicated, and adjust the declarations of `std::ranges::minmax_element` in 24.7.8 [alg.min.max] to agree:

```
[…]
template<class ExecutionPolicy, class ForwardIterator, class Compare>
  pair<ForwardIterator, ForwardIterator>
    minmax_element(ExecutionPolicy&& exec, // see 24.3.5 [algorithms.parallel.overloads]
                   ForwardIterator first, ForwardIterator last, Compare comp);

namespace ranges {
  template<class I>
  using minmax_element_result = minmax_result<I>;

  template<ForwardIterator I, Sentinel<I> S, class Proj = identity,
```

```
            IndirectStrictWeakOrder<projected<I, Proj>> Comp = ranges::less<>>
    constexpr minmax_element_result<I>
      minmax_element(I first, S last, Comp comp = {}, Proj proj = {});
  template<ForwardRange R, class Proj = identity,
            IndirectStrictWeakOrder<projected<iterator_t<R>, Proj>> Comp = ranges::less<>>
    constexpr minmax_element_result<safe_iterator_t<R>>
      minmax_element(R&& r, Comp comp = {}, Proj proj = {});
}

// 24.7.9 [alg.clamp], bounded value
[…]
```

3182(i). Specification of `Same` could be clearer

Section: 17.4.2 [concept.same] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2019-01-05 **Last modified:** 2019-01-20

Priority: 0

View all issues with Tentatively Ready status.

Discussion:

The specification of the `Same` concept in 17.4.2 [concept.same]:

```
template<class T, class U>
  concept Same = is_same_v<T, U>;
```

-1- `Same<T, U>` subsumes `Same<U, T>` and vice versa.

seems contradictory. From the concept definition alone, it is *not* the case that `Same<T, U>` subsumes `Same<U, T>` nor vice versa. Paragraph 1 is trying to tell us that there's some magic that provides the stated subsumption relationship, but to a casual reader it appears to be a mis-annotated note. We should either add a note to explain what's actually happening here, or define the concept in such a way that it naturally provides the specified subsumption relationship.

Given that there's a straightforward library implementation of the symmetric subsumption idiom, the latter option seems preferable.

[2019-01-20 Reflector prioritization]

Set Priority to and status to Tentatively Ready

Proposed resolution:

This wording is relative to N4791.

1. Change 17.4.2 [concept.same] as follows:

```
template<class T, class U>
  concept same-impl = // exposition only
    is_same_v<T, U>;

template<class T, class U>
  concept Same = ~~is_same_v<T, U>~~same-impl<T, U> && same-impl<U, T>;
```

-1- [*Note:* `Same<T, U>` subsumes `Same<U, T>` and vice versa.—*end note*]